Transcript for DEFCON 2017 Talk: Hacking the Cloud (Gerald Steere & Sean Metcalf)

trimarcsecurity.com/hub-post/transcript-for-defcon-2017-talk-hacking-the-cloud-gerald-steere-sean-metcalf

Sean Metcalf December 3, 2017

Updated: Dec 13, 2024



"DEF CON 25 (2017) – "Hacking the Cloud" with Gerald Steere (@DarkPawh)"

Gerald Steere, Microsoft C+E Red Team DEF CON 25 (2017), Las Vegas, NV July 2017
Transcript (courtesy of Trimarc) Download the PDF version of this transcript.

Gerald: *Hello and good afternoon, Def Con. I'm Gerald Steere.*

Gerald: *Brief background on me. I'm @DarkPawh on Twitter, 10 years red teaming, pen testing, government private sector. I've been on the Cloud Enterprise red team at Microsoft since 2014. Spoke at BlueHat and BSides Seattle, and I spend most of my day breaking Azure, one of the largest networks in the world. It's a really fun thing to do, and we're going to talk a little bit about that today.*

Sean: *Trimarc, a security company. I'm a Microsoft-Certified Master (MCM) in Active Directory, one of about 100 in the world, and I've spoken at a bunch of conferences as well as DEF CON. I'm happy to be back again. I'm a security consultant and researcher. I post a lot of interesting Microsoft security stuff on ADSecurity.org.*

Gerald: *All right. So the cloud. We're going to go through a little bit of what you need to know about the basics, what's in it for you as an attacker, how you do recon in the cloud, how do you do some basic attacks, how do you get from on-premises to the cloud, how do you go back onto premises from the cloud, and some countermeasures, and then we'll walk through a bit of a demo scenario.*

So really, what do attackers care about? What's in it for me? Well, as a security professional, whether you're internal, red team, consultant, whatever, your client is using

the cloud whether they realize it or not. It might just be a third-party application, but they definitely are using it. Many of the traditional techniques we know do not work in this environment. The concepts are similar but require a new way of thinking for a new environment.

Can I go after the client's cloud deployments? Well, we're not lawyers. If you're a professional, you should have some of those definitely, but the answer is, in general, yes. Scope and access is going to be more limited in a cloud environment because it's somebody else's computer, after all. You need to take that in mind when you're planning your operations. You need to spell it out in your reporting and how you're dealing with it. Make sure the customer understands, and some providers require an approval process. We're going to walk through some of the large providers and what their requirements are.

Azure and AWS both require pre-approval via the account owner for attacking. Google Cloud did not require that based on our research. We all have standard rules. You can only attack your client's assets, only the ones in scope, not going after other customers' things, limited to their ownership, no denial of service. Very typical for red teaming and penetration testing experts, so it shouldn't be too much of a surprise.

One thing I'd like to call out: the Azure rules of engagement does allow attempts at breaking client isolation with the stipulation that any success is reported to their security team immediately, and there are bounty programs available as well. So even if you're not a professional red team or pen tester, you might want to look into cloud bounties for things like isolation escape, and that can be pretty lucrative as well if you manage to find one.

What do you need to know about the cloud to get started? Let's talk about accessibility modifiers. Much like programming, clouds have accessibility modifiers as far as what you can access. Public cloud is what most people think of with AWS, Azure, Rackspace, any of the big public providers which are available to anybody who wants to pony up the money to run their code or run their environments in those.

Private clouds are usually internal to a large company or organization where they provide resources to only their internal organizations or partners within an environment that's usually charged by the hour, cross charged or whatever.

Hybrid cloud is really becoming the common pattern nowadays where large companies will host some components on premises in their data center and then farm out others to public clouds or multiple clouds even for redundancy and uptime purposes. Hybrid clouds especially need to be considered by the attacker because these are going to provide your opportunities to pivot back and forth between environments.

Now, there's All the aaS service words. Albert Barron posted this Pizza as a Service Chart on LinkedIn a couple years ago, and it's really a good description of what you have. If you consider traditional on-premise as a pizza, you're making it at home. You're buying the

ingredients. You're making it yourself. You're serving it up. You are responsible for everything.

Then you have infrastructure as a service or your take n' bake. You've picked up the pizza from somebody. They've made it for you, but you're responsible for cooking it and serving it.

Then you have platform as a service where everything is done for you like delivery. Everything is handled. You just need to enjoy it. In this case in the cloud, this is where you provide code to run on somebody else's server very much like..it's called serverless a lot of times or websites or functions or app services, but all you're providing is the code and that's what you're responsible for.

Then there's software as a service where everything is outsourced and managed by the vendor. This is going to be your Google Docs, your Office 365, your Salesforce. As an attacker, this is the most limiting target because you can't go after the infrastructure that supports this in general, but it can be the most lucrative.

Our goal is to prove risk to our clients. If we can do that by accessing their data and just dumping everything, it doesn't matter that we can't attack the infrastructure. If we can grab an API key and pull that data anyways, then we've succeeded in showing the risk to the company.

Let's look at the cloud as an operating system unto itself. The same idea, different words. This particular set of slides came from an amazing talk by some of my coworkers, Sacha Faust and Andrew Johnson. It was given at Infiltrate. It deals completely with exploitation in a cloud environment. I highly recommend watching that if you get a chance, but let's look at this.

Rather than a single server, you have a service where everything is done for you. You're not considering about what box it's running on or anything. You're just asking for them to provide a service like a database, and then everything behind it is handled. Rather than a domain, you have an account or subscription, and everything is contained within that container. So rather than a domain admin, you have subscription admins, and this is really a key point I want you to take away from this.

Subscription or account admin or root in the cloud environment is equivalent to domain admin in a traditional domain. If you can own the account on that level, you can own all of the resources within the account. And we often have a lot of times with privilege assignment here where people who shouldn't have access, like a billing administrator, is just added as a subscription admin so that they can handle the billing when they really should be handled as more as just administrative with no access. So keep that in mind. If you take nothing else away from this talk, owning subscription is equivalent to owning the cloud domain.

Rather than passing hashes, we're looking for credential pivot. We're going to go into a lot more detail on this and various types of credentials used in cloud environments, but it's not just one type. There are many different types you need to consider.

Rather than private IPs, we have public IPs. It's like MAC had never been invented. This is awesome for attackers, not so great for defenders. Things are just out open to the public because that's where they're designed to be. They do implement VPNs and things like this, but in many cases, what you're after is a lot more accessible than it would be tucked into the crunchy center of somebody's well-protected data center.

And then rather than RDP and SSH, which you still have on IAS boxes as mode admin, you're generally dealing with management APIs where you're making requests to the service to do something on your behalf. These are often your target when you want to make a service or cloud do something for you.

But the real important question is: Where's the data? All cloud services rely on some type of data storage for nearly everything. Whether it's a bucket or storage account, when you have a virtual machine or a service, it's writing that data to an account that you can access with just a storage key or API key. You might not need to attack a virtual machine at all if you can just dump the drive for that machine out of a storage account and pull what you need out of it. As an attacker, you've got to look at what is your real goal? Do you need to get code executing on a box which might be well monitored, or can you just download a copy of it and run it yourself?

At this point, I'm going to hand it off to Sean to talk about ways to do some recon in the cloud.

Sean: *Hey DEF CON, how's it going? Excited to be back again. Let's look at recon in a cloud-type environment. You have a customer. They've hired you to come in and pen test, red team their environment, and they said, "We want to add cloud to the scope." What does that mean? How do we identify what sort of cloud services they have?*

Well, DNS is your best friend, just like it always has been. It's there. People put a lot of things into it to help resources and users find other services. Like MX records, we can find a lot of information from MX records. So I did some scanning of DNS on a bunch of companies to see what I could find, and I discovered that you can find interesting information in DNS, which we've known. But you can look for companies that are using Office 365, Google apps. They also have MX records for the specific security hosted email systems: Proofpoint, Cisco, Cyren, CSC. All those we can find by those MX records. What's interesting about this is when you find something like pphosted, which is Proofpoint, and then you find that there's a DNS TXT record for Office 365, that gives some interesting information about how their email security is configured.

At this point, you know they have Proofpoint or something else, and so when the email comes in, it's going through that security system and then being delivered to their mailbox. When you're designing your phishing campaign, you want to take that into account.

You can also find some other interesting things in the TXT records within DNS, such as if they're using Amazon Simple Email or their MDM system. I found a bunch of Symantec MDMs that are configured here, and they actually point to what that system is running on or if their website is running Azure websites. The thing that I thought was really interesting was Paychex, DocuSign, Atlassian, just to name a few of these cloud services, actually get registered and configured in DNS to identify this domain as a customer of that cloud service.

Then we can take it one step further and actually look at those SPF records. So the SPF records are obviously the mail-sending systems, those that are authorized to send on behalf of a domain. So we pair this information with the MX records and the DNS TXT records that I've already shown, and you get some good information about what cloud service providers are actually being used by this customer. Then you can look and see Salesforce, Mailchimp, etc., and some of these you could even leverage as part of your spear phishing campaign because obviously they're doing business with them. And if there are a couple of different subsidiaries in this company, you could actually leverage this for one of the subsidiaries to send to the main one in order to make that part of the phishing campaign, depending on what's in scope.

The other part of it which is pretty interesting is looking for federation servers because ultimately that's the key to authentication within the cloud. Since there's no standard naming for these federation servers, we can do DNS queries for a bunch of different A records, and what I found interesting as part of this scanning is that a number of these actually are registered as C names and then point back to what the federation server actually is. And we can do DNS queries for a number of these: ADFS, etc. STS seems to be one of the more legacy configuration for ADFS.

And then we can go look at this web server to see what the configuration is. We can look at the data in the webpage. We can look at the headers, which gives us information about what kind of server it's running. So if it's running IAS, it's probably an ADFS server or ADFS proxy. We can get some other information out of it as well such as how long those tokens are good for and some other things about what domains hook into that, and we can identify if they have exchange on premises. Well, we can look at that through the MX record, but we can look to see where their autodiscover is if they're using a Microsoft-based email system, which a lot are. And then we can identify the OS server that's there. We can also identify what version of Exchange OS is running by looking at this copyright banner. If it's copyright up to 2006, then it's Microsoft Exchange 2007.

How does this tie together? We have cloud. We have Federation. We need to combine these things somehow. Ultimately, it comes down to the authentication that's leveraged. We have authentication and authorization just like we've always had. Authentication is identifying and confirming that who you say you are you actually are, and then the authorization part is that component that says that you should be able to have access or you're a member of certain groups or you have certain attributes and you should be able to access this resource or potentially access that resource.

Ultimately, how that authentication happens depends on the cloud provider and the protocols they support, like OAUTH, openID, SAML, WS-Federation, WS-Trust. In a Federation world, ultimately the user authenticates locally within the on-premise environment, often in Active Directory. They get their Kerberos ticket, and they open up their web browser and hit the link to go to that cloud app or those multiple cloud apps. What ends up happening is that cloud app bounces them back to their Federation system because they don't have a token or they don't have that cookie with that information that the cloud app needs to actually determine whether or not they should have access and what level of access they should have.

So they hit that Federation server. The Federation server checks their identity, adds the information into that token such as claims and make some assertions about that user. Since that cloud app trusts that token because it's signed by that Federation server, that cloud app is going to make the decision as to what that user should have access to.

I'll talk about ADFS for just a minute here because the Microsoft Active Directory Federation Server (ADFS) is pretty common, pretty widespread in environments that are leveraging Microsoft services like Active Directory and certainly if they're leveraging Office 365 or Azure. The ADFS servers are going to be inside the network joined to the domain. That's key. There's going to be a proxy server that's going to be out in the DMZ or directly connected to the internet. With earlier versions of ADFS, you're actually looking at an ADFS proxy. With the newer versions, you're looking at a web application proxy that handles those requests coming in.

This ADFS server is going to have three different types of certificates installed. The service communication for that initial HTTPS communication into the system and then some token decrypting and signing certs. Now, these last two may actually be the same certs. They are often internal certs whereas the service communication cert is often one from a CA trusted by the internet.

In ADFS, the organization is going to set up a relying party trust which is that cloud service and application that they want to actually federate to and have a trust with. Then the claim rules are interesting because organizations can actually lock down how access occurs. So they can say, "If we have Office 365, our users have to authenticate from within our network on our domain and are not allowed to go from the internet at large directly into that

Office 365 environment. We can force that through the Federation environment." Often don't, but it can be done.

SAML is kind of what ties all this together, and most of the time, this is what's used when we're talking about Federation–Security Assertion Markup Language (SAML). It's basically just to support this web browser single-sign on, and there are three roles. This should be very similar to what people are used to when you're going after an Active Directory environment. Right? You have your user, your identity provider, and your service provider. Just like in AD, you have your user. You have your KDC or your domain controller, and you have some Kerberos service running on a server.

What ends up happening is user goes to the Federation server or the identity provider and proves their identity, gets those claims and that token, and then can access that service provider just like in Kerberos. SAML in this instance is specifying the assertions that are handling between these roles or what's required in order for that service provider to identify that user and know who they are. So it's providing a broker-type service.

Now, SAML is authentication method agnostic, so it could be Active Directory. It could be LDAP. It doesn't really matter. The key to this is the SAML messages are leveraging these certificates, these signatures. The SAML message itself is signed, and there are a number of certificates or signing that occurs inside to ensure the trustedness of the whole system.

The key point here is that the certificates really matter. Certificates on a Federation server matter because those are what are used to sign that token and the components in that token to prove to that service provider that, yes, this user is who they say they are and these are the attributes they have. These are the claims that they have. Then you can use that to determine whether or not you should give them different types of access.

Like I said, SAML is really what happens a lot on the internal network. The server that is going to be connected to from the internet is running HTTPS. It has to be available in order for users that are connecting in from the internet in order to use Federation and connect to these cloud resources.

Here's where it gets interesting. This Federation is effectively like a cloud Kerberos. Just like with the domain controller where the KRBTGT is the thing that signs all of those Kerberos tickets, on the Federation server, those certificates are what sign those tokens that say that these users should have access. If it's possible to actually extract those certificates off of that Federation server or potentially find them elsewhere as Casey (@SubTee) mentions, that means you can spoof access to any cloud service you want provided that that organization has a trust with that. So we're looking at kind of a golden ticket in the cloud environment. Ultimately, by stealing this and using Mimikatz to actually export that certificate on the cloud server, we could get a lot of cloud access.

When we're talking about on-premise cloud components, we're looking at Active Directory, right? Provide single-sign on for users. They can then connect to the cloud services free, intuitively, easily without having to log on again, but what's interesting about this is most of these cloud services require some sort of synchronization from the Active Directory environment into that cloud service. A lot of times they sync all of the users and all the attributes into that cloud environment. IT for that organization may not even be aware of this because it just requires a regular user account. A lot of times many organizations don't even know all the cloud services that are actually in their environment.

Let's look at , which is used as a sync tool in order to synchronize those user accounts in groups and etc. into Azure. What's interesting and what I want to call out here is that a lot of organization click the easy button. They click Express Permissions, which just runs through and sets everything in AD for them. Well, part of this is there's a component here I want to call out called Replicate Directory Changes and Replicate Directory Changes All. Does anyone know what that's for or how that could be used for an attack? DCSync, right? Those are the rights that are required in order to do DCSync. So that means the service account actually has these rights if they've hit Express Permissions, even if that organization is not using password sync feature. The password sync feature is where that password hash is hashed again and then sent up to the Azure AD environment, and that way users can actually log in without going through the on-premise environment.

Now, if they click the Custom Permissions button, then they can go through and select these individually. But even if they're maybe thinking about this password sync, they might have these rights that are configured. So as you're operating in an AD environment where they're looking at using Azure or Office 365, you might notice if there's an Azure AD account that actually has these very high-level rights.

When we're talking about cloud stuff, we're still talking about PowerShell. All of the major cloud providers have their own PowerShell modules (Amazon AWS, Google Cloud, Microsoft Azure, Microsoft Office 365), and they're really useful for administrators, probably for other things. Once we have an account we can leverage, because remember those accounts inside that organization typically have cloud access, cloud permission, we can get information about that company as configured..here I'm using Office 365 as an example and using that O365 PowerShell module. We get information about how they're configured, what the directory synchronization information is, if they're actually doing password synchronization. If you find that their account actually has those Replicate Directory Changes rights but this is not set, then you can let them know that, hey, you're over-permissioning this account.

We can get those Office 365 roles. We can numerate what the membership is, and then we can identify those user accounts on their on-premises environment to identify what access they have to their cloud environment. The other thing we can do is we can find service accounts in their cloud environment. Why this is interesting is we can start looking

at these URLs that are identified as part of the service principal names, and we can identify that there is a DinoDNA.ingentech.co. There's a Secure.ingentech.co, and there's a unixSystem.ingentech.co. There's associated groups with these, and these groups have accounts in them.

As we're looking at their cloud environment, we can start looking at these applications that are configured within their cloud environment and see if there's some interesting information there. In Office 365 and in Azure, Microsoft has a REST API that allows access, and it enables you to enumerate a lot of data about Azure AD. So on the on-premises environment you have LDAP. In this Azure AD cloud environment, you have a REST API, and the graphics on this webpage actually shows the type of information you can get. You can do like PowerView-type stuff in Azure AD through this website.

So when we're talking about cloud assets and looking to see what they have, it's important for organizations to understand that managing their VMs are still their responsibility. A lot of these cloud providers provide Quick Start installation scripts, things that set up the VMs very quickly, configure them for that environment, but they may not be secure. They usually aren't. You want to check that and identify what those are. Get comfortable with what these scripts are and how Azure has them, AWS has them, or any cloud provider has them. Look at those scripts. Know what those are before you go into that environment.

Microsoft had docs.com where customers could actually upload and put data and documents into docs.com and then share them with their employees, share them with customers. The problem was that a lot of people clicked too many times and allowed it world readable, which meant that a lot of sensitive documentation was available here. Kevin Beaumont (@GossiTheDog) actually highlighted this in Twitter and said, "Hey guys. There's a lot of information here, and guess what, it has a search feature. I can find all this through docs.com." Microsoft said okay, we're going to remove the search feature. There's still Google. Kevin says, "Hey guys, I can still see this stuff." So Microsoft went back, did some changes, shut it down for a little while, brought it back up, and said, "Customers, please check your stuff. Make sure you're not sharing your sensitive documents with the internet."

But it's not just Microsoft. Just a few weeks ago, we had Amazon that had some issues with S3, right? By these headlines, we would think the cloud is a problem. Cloud security failure, data exposure. Like the sky is falling, the cloud is burning, right? Not really. Some more reasonable headline is human error because that's what it is.

So if you're hired to evaluate the environment and the cloud assets, definitely look at their Amazon S3 environment, look at docs.com. Look at these things to see what you can find and help them identify these potential exposures before they get exposed by someone else. Amazon actually emailed their customers and said, "By the way, you have some data you're sharing on your Amazon S3. You probably want to look at that."

There was a great blog article on Detectify.com about how these configurations were configured and how to fix them. The most interesting part of this for pen testers is there's an API to actually look for this data. So you definitely want to read this article and kind of dig into that.

Now I'm going to turn it back to Gerald who's going to talk to you about how credentials are used in the cloud.

Gerald: *We've got all the hashes. We have KRBTGT, but your cloud provider really doesn't care. As Sean was just talking, they speak a different language. They speak Federation. You might be able to get in if the Federation server that provides that permission is on your domain, but you might not.*

So what do we do? Well, creds never change. They're always good. The type of cred changes though. In the cloud world, we're mostly interested in certificates and private keys and API tokens. What has this done? Well, it's made popping dev boxes more productive than ever. I've pretty much made my career on stealing secrets off of dev boxes for internal lateral movement because nobody else paid any attention to the web configs that were on open shares and networks a lot of times.

Well, now we have all kinds of things. We have private certificates, which might be checked in the repos, and we have these API keys. There's even more of these secrets. And you do know Mimikatz can export certificates, right? The non-exportable flag in Windows is more of a recommendation than a requirement, and it's pretty easy to bypass that. If there's any of my Windows colleagues in the audience, I would really like to see this moved to the Cred Guard Secure Boundary at some point so people will stop exporting non-exportable certificates.

And what is old is new again: password spraying. Hey, this is like for us old people. It's almost like being back in the war dialing days of trying a few passwords and moving on. I mentioned earlier everything is on the public network now. We can do a little bit of spraying across their accounts and come back and it's API. So this is really easy to do. You write a four-loop and call the API. There are some tools in the references that do this as well. We don't have personal experience with those. Feel free to look it up. But this is a really good example of taking techniques you already know and reapplying them to new environments.

DevOps–they probably have what you're looking for. If you are not pillaging the internal source repos and the shares on developers' workstations, you're doing a pretty poor job as a red teamer and pen tester because they are usually loaded with juicy secrets, or maybe they just checked them in the public GIT as Sean showed earlier. That's not really good either.

How are the deployments done? Most cloud services are done through a continuous integration model where everything just gets packaged up and published out constantly

every time someone checks in a change. Well, if I'm on the developer's box, can I just ride along on that check-in and get myself into production without ever even compromising a secret? It's another way you can trade in network access for access somewhere else.

As I mentioned, keys are everywhere. Leaks into Github have been super common. People leave their access keys on their desktops. Download folders are a great place to dump dev boxes. What do we do with all this though? What's our access? If we have on-premises access but our real target is data, let's find a way to get cloud access and trade that in.

Very similar you have cloud assets that need to talk to things on premises. Well, there's probably a data path you can follow. This is one of the key things to being a good attacker is identifying all the possible data paths. Ask yourself how is this system communicating because that is often going to tell you where you need to go to achieve your objective. And really, is there shared authentication methods? People still love sharing passwords. If you have an account that provides cloud access, you've got their hash, and you can crack their hash, then who knows? I might have just reused the same account for a live ID or an Amazon account that they're using to manage their entire companies domain. That's something worth trying, and it works more often than it should.

At the end of the day, we are here to provide value and service to the people that hired us to attack their stuff. I love my job. It's fun, but really my ultimate goal is making sure things are more secure. So what do we do about all this?

First and I think most important when it comes to cloud environments is probably managing credentials and secrets. Most of the big clouds all provide some type of automated credential store, and if you and your clients are not using these, you need to learn how because this is one of the best ways to handle it. There are so many different credentials in the cloud. Make it easy for people to do the right thing.

MFA (multifactor authorization) is huge because a lot of the times your cloud identity is the same as identity for another account. Make it mandatory. And for things like SPNs that can't have MFA, make sure they have as little access as possible and that you monitor it as much as possible for any deviation.

Reviewing permissions, as Sean mentioned. A lot of stuff gets made public that's never meant to be. That's a really big issue. Check your VPN access because things being open on the network has become the default again, opening the internet, but that's not necessarily a good thing. A lot of times you can just ask does it really need to be open, and the answer is no. These providers provide ways to limit this, but it's just not used a lot of times.

Least privilege and least access is still your best friend. All the clouds provide methods for managing these permissions which Sean will cover a little bit in a moment. The idea of

even like a secure admin workstation for your domain controllers and your privileged access just the same for your cloud admins. As I mentioned, they are functionally equivalent to domain admin in the cloud, so they need to have the same sort of protections. If they are logging into their cloud account with the same account they check their email with, they're just a phish away from your entire cloud environment being under somebody else's control.

Credential management is absolutely key because there's so many of them, and you have to keep track of them. You have to roll them. You have to be able to do this in an automated way. This is a huge opportunity for attackers, and it's not so great on defenders because a lot of times they aren't necessarily easy to handle even if you know they've been exposed.

Sean: *So when it comes to Federation, there's a few things that are important to do. We want to make sure that Federation servers are protected at the same level as domain controllers. That proxy server is important to prevent that communication from coming in from the outside. Auditing that cloud authentication. Getting that logging to make sure that the info security team actually knows what sort of authentication is occurring so if there's something that looks weird they know what that is.*

I'm going to cover some more recommendations, and then Gerald is going to do a demo so we can see kind of what all this looks like put together. Multifactor authentication is important, certainly for these admin accounts. Anyone that manages those Federation servers probably should have MFA configured.

As I mentioned, we can protect and control that cloud authentication via Federation rules. So in this instance we can ensure that users internally maybe they're a little more protected. They can connect into their cloud services through a single-sign on. They don't need anything more than their username and their password, but if they're coming from the internet, they have to use two-factor.

We can also leverage those cloud provider security features and definitely recommend to your customers that they do this. Azure and Amazon provide great ways to monitor what's going on to identify what resources are there and which keys have access to them, and you can actually restrict that. So even if an API key gets leaked out on Github, nothing can be done with it because it's constrained in how it can be used and only from specific cloud assets.

Then of course, monitoring and learning is really important. We need to make sure when we tell our customers, here are our recommendations, you're monitoring. It's not your network, but you still have to have a good understanding of it. There are different tools from different cloud providers and how this can be done. Amazon provides VPC Flow, which is basically like NetFlow except to another level.

Defenders need to be familiar with all these tools as well and know that they can do that and know that these events that are gathered by the cloud provider can actually flow into their SIEM or their central logging tool. Of course, asset inventory is even more critical on the cloud because new VMs can be spun up all the time. And of course, every organization should assume that there's some sort of breach, and that's what we're going to talk about now.

Gerald: *All right. I've been hired to hack SithCo. Fairly easily got domain admin on one of their subsidiary domains for an offshore developer via some phishing, but so far, their corporate network has actually been fairly well protected. We did some recon and found that they host a bunch of websites out into the public cloud resource.*

So how do we leverage this access to get to corporate? Well, we're going to start with our internal access, do some recon, figure out where we are, pivot through the cloud, and eventually end up inside the corporate network.

Here's me running my favorite meterpreter on a developer box. The first thing I'm going to do is pillage their download directories because when you go to a website and ask to download your keys or your publish settings file, which is basically a self-contained, all-in-one pwn Azure file because it has your private certificate in it, they just dump into their downloads folder and usually forget they're there afterwards. So let's go ahead and try that.

RootKey is the AWS very similar version. It's an API key that gives full access, and it comes in a nice, handy plain text file. So yeah, these problems are not cloud specific. They occur across all clouds.

Well, here's a Publish Settings file. I'm using a really handy third-party tool called Azure Management Studio to make this easy to visualize. Normally I'd just be doing this all through the API. But oh no, this Publish Settings file is out of date. I no longer have access, but it's not a loss. They might have changed the certificate there, but I gained a piece of information I'll need later, which is a subscription ID, which does not change.

So we're still on the developer box. Let's see how they're communicating to Azure. Let's dump their certificate store. Well, they have a couple of Windows Azure tools encryption. Those are generated by Visual Studio. There's something called Azure Automation, which sounds pretty interesting. They'll usually have common tools depending on what chain was used to make it. Like Windows Azure Tools is a super common one. Like I said, learning these for your cloud environment and what they look like is super important.

I'm going to elevate to System and go ahead and patch the CryptoAPI so I can bypass that annoying no-export flag, and then I'm going to dump the certificate. Here's a quick example of, in this case, I got the public key and then immediately following got the private

key. If I hadn't done the patch step first, I would have only gotten the public key because that is a non-exportable certificate, which like I said is more of a suggestion.

So then I can take that certificate, convert the Base64 into a PFX file, and go back to Azure Management Studio. I got the subscription ID out of the Publish Settings file which was no longer valid. I now have a certificate on the box that may or may not work, but if it's a current certificate on the developer's box, there's a good chance it probably does. Let's go ahead and successfully authenticate to Azure and see they're running a couple of systems services.

Sithweb looks really interesting. Let's take a look at this VM. As I mentioned before, a lot of times we don't need to compromise the VM to achieve what we want to achieve. Let's start with the storage. This VM is running. I don't want to interrupt things which might get detected, so I'm just going to take a snapshot of the disk, download it to my local hard drive and start pillaging it like I normally would. If you're familiar with .NET apps at all, Web.config is your favorite friend.

I'm going to pull this down and start looking for some interesting stuff. In this case, it has access to a SQL server as SSA in a private network space, which indicates there's probably a VPN involved somewhere. It could just as easily be to a public IP, but a lot of times this one was at least partially set up correctly. They put a VPN in place so they didn't expose their SQL server to the public, but it is exposed to their cloud server. Thankfully, they just ran SSA. That makes our lives a little bit more easy.

At this point, we probably need to get code running on this box to access this VPN. A lot of bigger companies have direct VPN access to their cloud assets through a direct switch where they set up a permanent link to their cloud environment. If I were to get code running on this box and I have subscription level access, which I said is basically equivalent to domain, I could hit up MSDN real quick and see there's an API for resetting the password on any VM I want. Well, that's great. There's a good chance this will get caught if the monitoring team is doing the job, but if this was set up by some devops team that didn't consult their defenders, there's a good chance nothing on the VM is being monitored. That's really important to consider is that is what needs to be fixed in the cloud world is taking these into account.

I'll go ahead and enable RDP on NetBox, log in, and here's just a..I would normally have gone in through, launched a meterpreter through RemoteShell, but I just want to show real quick. I now have direct access to the corporate SQL server via the VPN link from this VM which we compromised from a domain that was completely untrusted by pivoting through their cloud assets. This is going to be a fairly common practice in the future. It's a good example of a hybrid cloud environment where they're sharing data back and forth, and it absolutely needs to be considered going forward.

I'm just about out of time, so I want to say there is going to be a lovely narrated video up on ADSecurity.org following the presentation of the entire end-to-end attack chain. I hope you'll enjoy that. It would have been boring just sitting here watching it in the talk rather than us giving you advice.

Sean: Just to summarize what Gerald showed, we had a subsidiary that was completely separate in their own Active Directory environment, and they did development for this cloud environment. Gerald compromised that developer box, pivoted to the cloud, got onto one of the cloud assets, saw that that web server actually had credentials for the corporate SQL server inside their corporate network, pivoted from that subsidiary untrusted environment through the cloud into the corporation, and now has moved into that corporation and continues to move and plunder. Great job, Gerald.

Gerald: *Thank you all very much.*

Sean: *Thank you all. Appreciate it.*

Transcript for Gerald Steere & Sean Metcalf's talk at DEFCON (Las Vegas, NV) in July 2017. Download the PDF version of this transcript.

Copyright © 2017 Trimarc

Content may not be reproduced or used without express written permission.